

Issue #257: Kaggle 双 GPU 验证记录

本文记录 Issue #257（训练运行期间记录 GPU 显存、利用率和温度历史）在 Kaggle 双 GPU 环境中的端到端验证过程与结果。

验证结论

PASS。在 2 张 Tesla T4 上使用 TP=2 完成 2588 个 SFT 训练 step 后：

- 训练正常完成，没有因 GPU 监控引入中断；
- 两张 GPU 均持续产生显存、利用率和温度样本；
- 有界 JSONL 历史和 JSON 汇总文件均成功生成；
- 训练结束时正确输出双 GPU 可读摘要；
- 最终保留 1000 行历史记录，符合 `--gpu-stats-history 1000` 的总行数上限。

测试版本

- 分支：`feat/issue-257-gpu-stats`
- 提交：`e17c3ab9cb819db96ead78bab6a6fab9859c8b58`

环境

项目	配置
平台	Kaggle Notebook / Linux x86_64
Python	3.12.13
PyTorch	2.10.0+cu128
CUDA	12.8
GPU	2 × Tesla T4, 15360 MiB/卡
Compute Capability	7.5
并行配置	<code>world_size=2</code> 、 <code>tp_size=2</code>

两张 T4 不支持本次配置所需的 Flash Attention 路径，因此训练使用 `native attention` 兼容路径；这不影响 Issue #257 的 GPU 指标采集验证。

安装

在仓库根目录执行：

```
import os

os.chdir("/kaggle/working/AReno-issue257")
```

```
pip install psutil -q
pip install flash-linear-attention -q
pip install -e . --no-build-isolation
```

安装后，`areno check` 能够识别两张 Tesla T4，`areno_accel` 和 `flash_linear_attention` 均可导入。

数据集准备

为控制训练时长，从 `yahma/alpaca-cleaned` 的训练集读取前 10%，并导出为本地 JSONL：

```
from datasets import load_dataset

dataset = load_dataset("yahma/alpaca-cleaned", split="train[:10%]")
dataset.to_json(
    "/kaggle/working/alpaca-cleaned-10pct.jsonl",
    orient="records",
    lines=True,
    force_ascii=False,
)

print(len(dataset)) # 5176
```

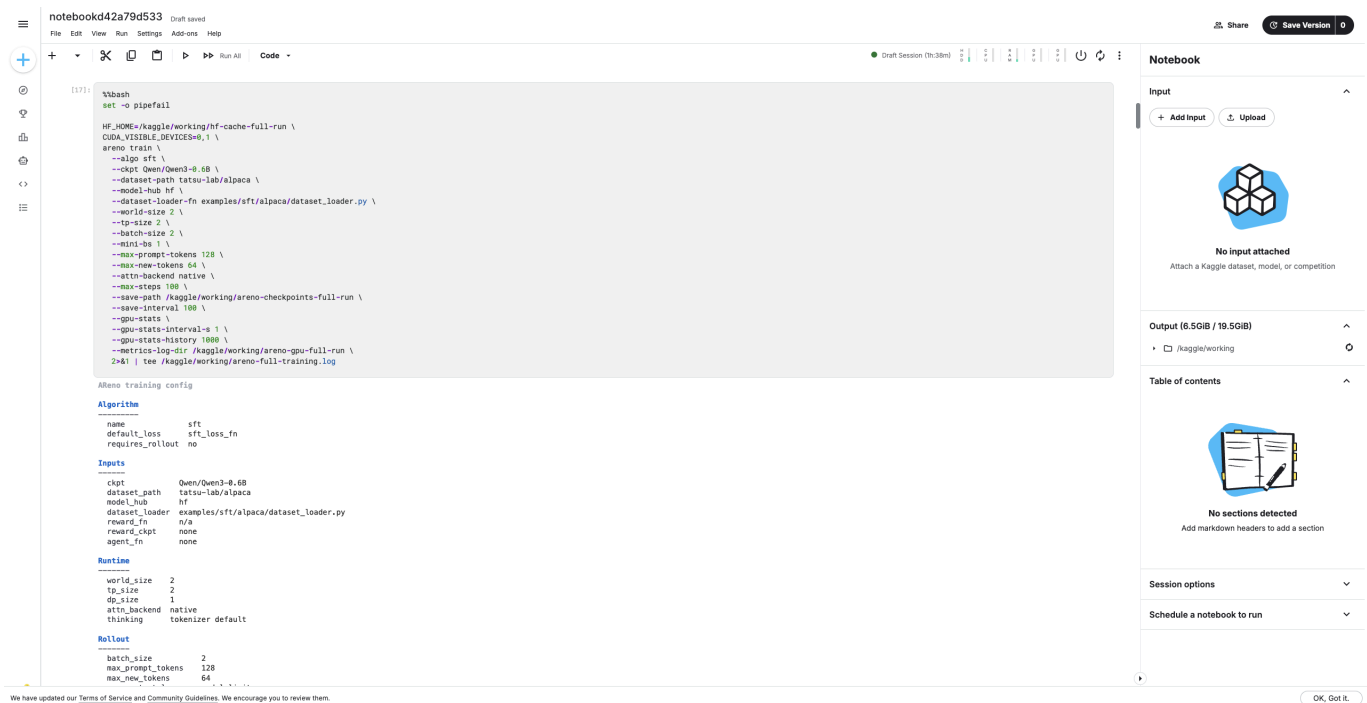
在 `batch_size=2`、`epochs=1` 的配置下，预期训练步数为：

```
5176 / 2 = 2588 steps
```

预备 smoke test

长训练前先使用 Qwen3-0.6B 完成 100-step 双 GPU smoke test，确认：

- `--gpu-stats` 能够启用；
- TP=2 训练可以启动；
- 采样线程不会阻塞训练；
- 训练配置中正确显示 GPU 统计参数。



长训练命令

```
areno train \
  --algo sft \
  --ckpt Qwen/Qwen3.5-0.8B \
  --model-hub hf \
  --dataset-path /kaggle/working/alpaca-cleaned-10pct.jsonl \
  --dataset-loader-fn examples/sft/alpaca/dataset_loader.py \
  --world-size 2 \
  --tp-size 2 \
  --epochs 1 \
  --batch-size 2 \
  --mini-bs 2 \
  --max-new-tokens 1024 \
  --save-path /kaggle/working/qwen35-checkpoints \
  --save-interval 1000 \
  --gpu-stats \
  --gpu-stats-interval-s 5 \
  --gpu-stats-history 1000 \
  --metrics-log-dir /kaggle/working/qwen35-metrics
```

Kaggle 默认暴露两张 GPU，因此本次运行不依赖显式设置 `CUDA_VISIBLE_DEVICES`。如需固定设备，也可以在命令前设置：

```
CUDA_VISIBLE_DEVICES=0,1 areno train ...
```

其中：

- `gpu_stats.<pid>.jsonl`：有界的逐设备采样历史；
- `gpu_stats_summary.<pid>.json`：本次运行的逐设备汇总；
- 其他文件为 AReno 已有的运行配置和 TensorBoard 指标产物。

```
[28]: !find /kaggle/working/qwen35-metrics \
      -maxdepth 1 \
      -type f \
      -printf "%f %s bytes\n" \
      | sort

areno_run_config.1782.json 5975 bytes
areno_run_config.1782.txt 1659 bytes
dashboard_state.1782.json 149 bytes
events.out.tfevents.1785246491.e76e6328a95.1782.0 4416622 bytes
gpu_stats.1782.jsonl 215715 bytes
gpu_stats_summary.1782.json 777 bytes
```

JSONL 内容检查

使用下面的代码读取历史文件，并查看当前保留窗口的首尾采样：

```
import json
from pathlib import Path

root = Path("/kaggle/working/qwen35-metrics")
history_path = next(root.glob("gpu_stats.*.jsonl"))

rows = [
    json.loads(line)
    for line in history_path.read_text().splitlines()
    if line.strip()
]

print("文件: ", history_path)
print("总行数: ", len(rows))
print(json.dumps(rows[:2], indent=2))
print(json.dumps(rows[-2:], indent=2))
```

```
> import json
from pathlib import Path

root = Path("/kaggle/working/qwen35-metrics")
history_path = next(root.glob("gpu_stats.*.jsonl"))

rows = [
    json.loads(line)
    for line in history_path.read_text().splitlines()
    if line.strip()
]

print("文件: ", history_path)
print("总行数: ", len(rows))

print("\n=== 最早一轮采样 ===")
print(json.dumps(rows[:2], indent=2))

print("\n=== 最后一轮采样 ===")
print(json.dumps(rows[-2:], indent=2))
```

检查结果显示：

- JSONL 总行数为 1000；
- 每个采样周期分别记录逻辑设备 0 和 1；
- 记录包含时间戳、设备编号、物理编号、UUID、型号、显存、利用率和温度；

- 首尾记录均能正确对应两张 Tesla T4。

文件: /kaggle/working/qwen35-metrics/gpu_stats.1782.jsonl
总行数: 1000

=== 最早一轮采样 ===

```
[
  {
    "timestamp_s": 1785246811.2441282,
    "index": 0,
    "name": "Tesla T4",
    "mem_used_mb": 11777,
    "mem_total_mb": 15360,
    "util_pct": 84,
    "temp_c": 78,
    "physical_index": 0,
    "uuid": "GPU-0abdd28-6145-28af-92f1-bfdaaac648e"
  },
  {
    "timestamp_s": 1785246811.2441282,
    "index": 1,
    "name": "Tesla T4",
    "mem_used_mb": 11777,
    "mem_total_mb": 15360,
    "util_pct": 88,
    "temp_c": 77,
    "physical_index": 1,
    "uuid": "GPU-ac6874ce-0bdd-2d21-b89e-42d201304e42"
  }
]
```

=== 最后一轮采样 ===

```
[
  {
    "timestamp_s": 1785249338.988179,
    "index": 0,
    "name": "Tesla T4",
    "mem_used_mb": 11907,
    "mem_total_mb": 15360,
    "util_pct": 91,
    "temp_c": 77,
    "physical_index": 0,
    "uuid": "GPU-0abdd28-6145-28af-92f1-bfdaaac648e"
  },
  {
    "timestamp_s": 1785249338.988179,
    "index": 1,
    "name": "Tesla T4",
    "mem_used_mb": 11909,
    "mem_total_mb": 15360,
    "util_pct": 93,
    "temp_c": 77,
    "physical_index": 1,
    "uuid": "GPU-ac6874ce-0bdd-2d21-b89e-42d201304e42"
  }
]
```

汇总结果

```
{
  "interval_s": 5.0,
  "max_history": 1000,
  "n_samples": 1000,
  "duration_s": 2527.744,
  "devices": [0, 1],
  "device_selectors": ["0", "1"],
  "reason": null,
  "failure": null,
  "per_device": {
    "0": {
      "name": "Tesla T4",
      "peak_mem_used_mb": 12481,
      "mem_total_mb": 15360,
      "mean_util_pct": 90,
      "max_temp_c": 78,
      "n_samples": 500
    },
    "1": {
      "name": "Tesla T4",
      "peak_mem_used_mb": 12483,
      "mem_total_mb": 15360,
      "mean_util_pct": 91,
      "max_temp_c": 78,
      "n_samples": 500
    }
  }
}
```

`--gpu-stats-history 1000` 是所有设备记录的总上限。由于本次使用两张 GPU，每个采样周期产生两行，因此最终每张 GPU 各保留 500 个样本。`duration_s` 和聚合值描述的是当前保留的有界历史窗口，而不是已经被淘汰的更早样本。

验收项

验收项	结果
双 GPU 设备识别与映射	PASS
显存、利用率和温度采样	PASS
训练过程不中断	PASS
2588 steps 长训练完成	PASS
历史记录总量有界	PASS
JSONL 历史文件生成	PASS
JSON 汇总文件生成	PASS
终端可读摘要输出	PASS

最终结论： **Long training and GPU stats validation: PASS。**